PRACTICAL - 05

Aim: Shape areas (Shape.java + Driver.java)

Code Screenshot:

Visual Studio Code - Shape.java

Shape.java

```java
public abstract class Shape {
    abstract double area();
}

class Circle extends Shape {
    double radius;
    Circle(double radius) { this.radius = radius; }
    double area() { return Math.PI * radius * radius; }
}

class Rectangle extends Shape {
    double length, width;
    Rectangle(double length, double width) {
        this.length = length;
        this.width = width;
    }
    double area() { return length * width; }
}

class Triangle extends Shape {
    double base, height;
    Triangle(double base, double height) {
        this.base = base;
        this.height = height;
    }
    double area() { return 0.5 * base * height; }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Shape[] shapes = {
            new Circle(3), new Rectangle(4, 5), new Triangle(3, 4)
        };
        double total = 0, largest = 0;

        for (Shape s : shapes) {
            double a = s.area();
            System.out.printf("%s: %.2f%n", s.getClass().getSimpleName(), a);
            total += a;
            if (a > largest) largest = a;
        }
        System.out.printf("Total: %.2f%nLargest: %.2f%n", total, largest);
    }
}
```

Output Screenshot:



Learning Outcome:

- Understood abstract classes and abstract method declarations.
- Mastered runtime polymorphism and dynamic method dispatch.
- Learned to process heterogeneous collections using base-class references.
- Implemented aggregation and extremum tracking in polymorphic loops.
- Understood the Open-Closed Principle (OCP) using class hierarchies.

Aim: Payroll (Employee.java + Driver.java)

Code Screenshot:

Visual Studio Code - Employee.java

Employee.java

```java
public abstract class Employee {
    protected String name;
    protected int id;

    Employee(String name, int id) {
        this.name = name;
        this.id = id;
    }
    String getName() { return name; }
    abstract double monthlySalary();
}

class FullTime extends Employee {
    double salary;
    FullTime(String name, int id, double salary) {
        super(name, id);
        this.salary = salary;
    }
    double monthlySalary() { return salary; }
}

class PartTime extends Employee {
    int hours;
    double rate;
    PartTime(String name, int id, int hours, double rate) {
        super(name, id);
        this.hours = hours;
        this.rate = rate;
    }
    double monthlySalary() { return hours * rate; }
}

class Intern extends Employee {
    double stipend;
    Intern(String name, int id, double stipend) {
        super(name, id);
        this.stipend = stipend;
    }
    double monthlySalary() { return stipend; }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Employee[] staff = {
            new FullTime("Asha", 101, 30000),
            new PartTime("Neel", 102, 40, 200),
            new Intern("Riya", 103, 5000)
        };
        double total = 0;

        for (Employee e : staff) {
            double salary = e.monthlySalary();
            System.out.printf("%s: Rs. %.2f", e.getName(), salary);
            if (e instanceof Intern) System.out.print(" (Intern)");
            System.out.println();
            total += salary;
        }
        System.out.printf("Total: Rs. %.2f%n", total);
    }
}
```

Output Screenshot:



Learning Outcome:

- Learned constructor delegation to superclasses using super(...).
- Understood method overriding and specialized behavior in subclasses.
- Learned how and when to use the instanceof type comparison operator.
- Implemented polymorphic payroll computation.
- Practiced clean code structuring with abstract base classes.

Aim: Media late fees (Media.java + Driver.java)

Code Screenshot:

Visual Studio Code - Media.java

Media.java

```java
public abstract class Media {
    String title;
    int daysLate;

    Media(String title, int daysLate) {
        this.title = title;
        this.daysLate = daysLate;
    }
    abstract double lateFee();
}

class Book extends Media {
    Book(String title, int daysLate) { super(title, daysLate); }
    double lateFee() { return daysLate * 2.0; }
}

class DVD extends Media {
    DVD(String title, int daysLate) { super(title, daysLate); }
    double lateFee() { return daysLate * 5.0; }
}

class Magazine extends Media {
    Magazine(String title, int daysLate) { super(title, daysLate); }
    double lateFee() { return daysLate * 1.0; }
}
```

Visual Studio Code - Driver.java

Driver.java

```java
public class Driver {
    public static void main(String[] args) {
        Media[] batch = {
            new Book("Java", 3),
            new DVD("Movie", 2),
            new Magazine("News", 4)
        };
        double total = 0;

        for (Media m : batch) {
            double fee = m.lateFee();
            System.out.printf("%s: Rs. %.2f%n", m.title, fee);
            total += fee;
        }
        System.out.printf("Total late fee: Rs. %.2f%n", total);
    }
}
```

Output Screenshot:



Learning Outcome:

- Designed an abstract hierarchy for different media types.
- Applied method overriding to calculate type-specific late fees.
- Used polymorphism to process a mixed array of media objects.
- Calculated individual and total late fees for returned media.
- Practiced extensible object-oriented design using abstract classes.